

```
In [ ]: import warnings
import pandas as pd
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=pd.errors DtypeWarning)

from loader import load_data
path="./data_1_1/data1_1"
typeData="csv"
normalized=True
y_column="event"

X,Y,data,all_mappings=load_data(path=f"{path}.{typeData}",y_column=y_colu
X=X.astype("float64")
```

```
In [ ]: # Category Based Feature Selection for Expert Opinion
categories=[]
cols_to_drop=[]

if(len(categories)==0):
    categories=[{"name": "ALL_COLS", "num":0, "columns": X.columns},]

X.drop(columns=cols_to_drop,inplace=True)
```

```
In [ ]: from plots import plot_histograms_grouped
plot_histograms_grouped(X,path)
```

Available Algorithms

Dropping Methods

- Drop columns with null values exceeding threshold
- Drop rows with missing values

Statistical Imputation

- Impute with **mean** or **median**
- Impute with **class-specific mean/median**

Forward & Backward Filling

- **Forward fill** (ffill)
- **Backward fill** (bfill)
- **Interpolate** missing values

Model-Based Imputation

- **Model Imputation** (predict missing values)
- **Iterative Model Imputation**
- **KNN Imputation** (k-nearest neighbors)

```
In [ ]: import warnings
warnings.filterwarnings("ignore", category=RuntimeWarning)

from clean_data import handling_missing_data
from test_models import train_and_evaluate

X_clean, Y_clean, best_algo_clean, best_acc_clean, acc_holder_clean, chan
    X, Y, train_and_evaluate
    )
print(f"Best imputation algorithm: {best_algo_clean} with accuracy: {best
print("All accuracies:", acc_holder_clean)
```

Taming Outliers

These functions help clean our dataset from outliers efficiently and selecting the best outlier detection algorithm.

Available Algorithms

- **IQR Method(1.5):**

- Take 1.5 times the IQR and then subtract this value from Q1 and add this value to Q3

- **LOF:**

- For any data object **q**, the **LOF score** is computed as the ratio of the **average local density** of its **k-nearest neighbors** to its **own local density** [25].

$$LOF(q) = \frac{\sum_{x \in N_k(q)} lrd(x)}{|N_k(q)| \times lrd(q)}$$

where the **local reachability density (lrd)** of **q** is given by:

$$lrd(q) = \frac{|N_k(q)|}{\sum_{x \in N_k(q)} \max(\text{dist}_k(x, D), \text{dist}(q, x))}$$

- **SP:**

- Employ a **scoring measure** based on the nearest neighbor (**k = 1**) within random sub-samples (**S ⊂ D**).

$$S_p(q) = \min_{x \in S} \text{dist}(q, x)$$

where **dist(q, x)** represents the distance between **q** and **x**.

- **iForest:**

- A **random split** is performed on a randomly selected feature.
- The partitioning continues until either:
 - Each node contains only **one data object**, or

- The tree reaches its **height limit**.

$$iForest(q) = \frac{1}{t} \sum_{i=1}^t l_i(q)$$

- **iNNe:**

- This method builds **hyperspheres** using all dimensions of the dataset. The **isolation score** of a data object **q** is defined as:

$$I(q) = \begin{cases} \tau(\eta_{cnn}(q)), & \text{if } q \in \bigcup_{c \in S} B(c) \\ 1 - \tau(cnn(q)), & \text{otherwise} \end{cases}$$

```
In [ ]: import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

from time_outlier import taming_outliers

X_tame, Y_tame, best_algo_tame, best_acc_tame, acc_holder_tame, changes_tam

print(f"Best Taming Outlier algorithm: {best_algo_tame} with accuracy: {b
print("All accuracies:", acc_holder_tame)
```

Normalization Algorithms

These functions help standardize or normalize datasets to improve model performance and feature scaling.

Available Algorithms

1. MinMaxScalarNorm

- **Description:**
 - Scales features to a fixed range [0, 1] using minimum and maximum values
 - Formula:

$$X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$$

2. RobustScalarNorm

- **Description:**
 - Scales features using median and interquartile range (IQR) to handle outliers
 - Formula:

$$X_{robust} = (X - Median(X)) / IQR(X)$$

3. ZScoreNormalizationNorm

- **Description:**

- Standardizes features to have zero mean and unit variance
- Formula:

$$X_{std} = (X - \mu) / \sigma$$

```
In [ ]: if(normalized):  
        from normalization import normalization  
        X_normalization, Y_normalization, best_algo_normalization, best_acc_n  
    else:  
        X_normalization=X_tame.copy()  
        Y_normalization=Y_tame.copy()  
        best_algo_normalization="Not Normalized"
```

Some Plots

1. Plotting Y Column With Pie Chart to View Balance Between Classes
2. Heatmap for Correlation
3. Histogram for After Normalized

```
In [ ]: from plots import plot_pie_chart  
plot_pie_chart(Y,path=path,title="Y Column Pie Chart")
```

```
In [ ]: # Plot Correlation Heatmap  
from plots import plot_heatmap  
plot_heatmap(X_normalization,Y_normalization,path)  
plot_histograms_grouped(X_normalization,path=f"{path}_after_")
```

Feature Selection Algorithms

These functions help select the most relevant features from a dataset to improve model performance and reduce dimensionality.

Available Algorithms

1. *SelectK*

- **Description:**

- Selects the top K features based on statistical tests (e.g., `chi2`, `f_classif`)
- Uses univariate statistical tests to score each feature

- **Parameters:**

- `k` : Number of top features to select
 - `score_func` : Scoring function (default: `f_classif`)
-

2. *L₁ Based*

- **Description:**

- Selects features using L1-regularized linear models (e.g., Lasso, Logistic Regression)
- Features with non-zero coefficients are selected

- **Parameters:**

- `estimator` : L1-regularized model (e.g., `LogisticRegression` , `Lasso`)
 - `threshold` : Minimum coefficient value for selection
-

3. *TreeBased*

- **Description:**

- Selects features based on importance scores from tree-based models
- Uses the `feature_importances_` attribute from tree estimators

- **Parameters:**

- `estimator` : Tree-based model (e.g., `RandomForest` , `XGBoost`)
 - `threshold` : Minimum importance score for selection
-

4. *RFE(RecursiveFeatureElimination)*

- **Description:**

- Recursively removes least important features using a base model
- Fits the model multiple times and eliminates features with the smallest weights

- **Parameters:**

- `estimator` : Model used to evaluate feature importance (default: `LogisticRegression`)
 - `n_features_to_select` : Number of features to retain
-

5. *Chi2*

- **Description:**

- Selects top K features using the Chi-squared statistical test
- Suitable for non-negative, categorical or count data

- **Parameters:**

- `k` : Number of top features to select
-

6. *VarianceThreshold*

- **Description:**

- Removes features with low variance, assuming low-variance features do not carry useful information
- Filters features below a specified threshold

- **Parameters:**

- `threshold` : Minimum variance required to keep a feature

```
In [ ]: import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

from feature_selection import feature_selection

X_feature, Y_feature, best_algo_feature, best_acc_feature, acc_holder_feature = feature_selection(X_feature, Y_feature, threshold)

print(f"Best Feature Selection algorithm: {best_algo_feature} with accuracy: {best_acc_feature}")
print("All accuracies:", acc_holder_feature)
```

Correlation heatmap After Feature Selection

```
In [ ]: plot_heatmap(X_feature, Y_feature, path)
```

Balance Sample

```
In [ ]: from sklearn.model_selection import train_test_split
from balance import check_and_balance

x_train, x_test, y_train, y_test = train_test_split(X_feature, Y_feature,
                                                    X_bal, Y_bal, used_method = check_and_balance(x_train, y_train, train_and_test))
```

Export Clean data with No Outliers, Normalized and Balanced

```
In [ ]: X_full = pd.concat([X_bal, x_test], axis=0).reset_index(drop=True)
y_full = pd.concat([Y_bal, y_test], axis=0).reset_index(drop=True)
full_data = X_full.copy()
full_data[y_column] = y_full
full_data.to_csv(f'{path}_balanced{"_and_normalized" if normalized else ""}.csv')
```

Model Training Algorithms

These functions train and evaluate machine learning models with hyperparameter tuning using **GridSearchCV**. Each algorithm runs multiple configurations, evaluates performance using a custom scoring function, and selects the best-performing model.

Available Algorithms

1. *LogisticRegressionBased*

- **Description:**

- Uses Logistic Regression with elastic net regularization for classification.
- Handles imbalanced datasets using `class_weight='balanced'`.
- Useful for both feature selection and baseline classification.

- **Parameters (Grid Search):**

- `penalty`:[l1 , l2 , elasticnet]
 - `C`:[0.001 , 0.01 , 0.1 , 1 , 10]
 - `solver`:[saga]
 - `max_iter`:[1000]
 - `l1_ratio`:[0 , 0.5 , 1]
-

2. *KNNBased*

- **Description:**

- Uses the K-Nearest Neighbors algorithm.
- Classifies data based on the majority class of nearest neighbors.

- **Parameters (Grid Search):**

- `n_neighbors`:[3 , 5 , 7 , 9]
 - `weights`:[uniform , distance]
 - `algorithm`:[auto , ball_tree , kd_tree]
-

3. *NaiveBayesBased*

- **Description:**

- Uses Gaussian Naive Bayes for classification.
- Suitable for continuous features and fast baseline performance.

- **Parameters (Grid Search):**

- `var_smoothing`:[1e-9 , 1e-8 , 1e-7]
-

4. *RandomForestBased*

- **Description:**

- Uses an ensemble of decision trees with bagging.
- Selects features based on importance scores and supports imbalanced datasets.

- **Parameters (Grid Search):**

- `n_estimators`:[50 , 100]
- `max_depth`:[None , 10 , 20]

- `min_samples_split:[ 2 , 5 ]`
-

5. *XGBoostBased*

- **Description:**

- Gradient boosting framework optimized for speed and performance.
- Supports imbalanced learning with `scale_pos_weight`.

- **Parameters (Grid Search):**

- `n_estimators:[ 50 , 100 ]`
 - `max_depth:[ 3 , 6 , 10 ]`
 - `learning_rate:[ 0.01 , 0.1 , 0.2 ]`
-

6. *LightGBMBased*

- **Description:**

- Gradient boosting framework optimized for speed and efficiency.
- Handles categorical features and imbalanced data well.

- **Parameters (Grid Search):**

- `n_estimators:[ 100 , 200 ]`
 - `learning_rate:[ 0.01 , 0.1 ]`
 - `num_leaves:[ 31 , 63 ]`
 - `max_depth:[ -1 , 3 , 5 , 10 , 20 ]`
-

7. *CatBoostBased*

- **Description:**

- Gradient boosting library that handles categorical features automatically.
- Does not require one-hot encoding.

- **Parameters (Grid Search):**

- `iterations:[ 100 , 200 ]`
 - `learning_rate:[ 0.01 , 0.1 ]`
 - `depth:[ 4 , 6 , 8 ]`
-

8. *GradientBoostingBased*

- **Description:**

- Classic gradient boosting implementation in scikit-learn.
- Suitable for smaller datasets and interpretable models.

- **Parameters (Grid Search):**

- `n_estimators:[ 50 , 100 ]`
- `learning_rate:[ 0.01 , 0.1 ]`
- `max_depth:[ 3 , 5 ]`

9. *NeuralNetworkBased*

- **Description:**

- Multi-layer Perceptron (MLP) classifier.
- Learns complex non-linear decision boundaries.

- **Parameters (Grid Search):**

- `hidden_layer_sizes:[ (50,) , (100,) , (50, 50) , (100, 50) ]`
- `activation:[ relu , tanh ]`
- `solver:[ adam ]`
- `max_iter:[ 300 ]`

10. *SVMBased*

- **Description:**

- Support Vector Machine (SVM) classifier with kernel support.
- Suitable for high-dimensional data and non-linear boundaries.

- **Parameters (Grid Search):**

- `C:[ 0.1 , 1 , 10 ]`
- `kernel:[ linear , rbf ]`
- `gamma:[ scale , auto ]`

```
In [ ]: import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

from model_training import model_training

model, best_algo_model, best_accuracy_model, acc_holder_model, cm, fpr, tpr = m

print(f"Best Model: {best_algo_model} with accuracy: {best_accuracy_model}")
print("All accuracies:", acc_holder_model)
```

```
In [ ]: from export import save_pipeline_summary
norm="normalized" if normalized else ""
save_pipeline_summary(final_results_feature, model, best_algo_clean, best_al
```

```
In [ ]: import joblib
import cloudpickle
import os
path_to_dir = path.split("/")
path_to_dir.pop()
new_path = "/" + path_to_dir

joblib.dump(model, os.path.join(new_path, "model.pkl"))
```

```
imputer = best_imputer_clean

with open(os.path.join(new_path, "clean_data.pkl"), "wb") as f:
    cloudpickle.dump(imputer, f)

joblib.dump(all_mappings, os.path.join(new_path, "all_mapping.pkl"))

joblib.dump(X_feature.columns.tolist(), os.path.join(new_path, "features."
```

```
In [ ]: from plots import calculate_p_values, format_p_values_table, SHAP, plot_conf
p_values_df=calculate_p_values(X_feature,Y_feature)
SHAP(X_feature,Y_feature,path)
plot_confusion_matrix_roc(fpr,tpr,cm,path)
format_p_values_table(p_values_df,path)
```